МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук
Кафедра автоматизированных систем управления

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к лабораторной работе № 1
«Создание экспертной системы с использованием прямой цепочки рассуждений»

Студент ПИ-23-1 Белозерцев Д.Р.
группа подпись, дата фамилия, инициалы

Руководитель
Первушин О.С.
ученая степень, ученое звание подпись, дата фамилия, инициалы

Липецк 2026 г.

Оглавление

Введение 4

1 Постановка задачи 5

2 Описание предметной области 6

2.1 Выбор предметной области 6

2.2 Принцип работы экспертной системы 6

3 Проектирование экспертной системы 7

3.1 Архитектура системы 7

3.2 Представление правил 7

3.3 Рабочая база данных 8

3.4 База знаний 8

4 Разработка программной части 10

4.1 Структура программы 10

4.2 Модуль управления правилами 10

4.3 Загрузка и сохранение базы правил 10

4.4 Алгоритм прямой цепочки рассуждений 11

4.5 Главное меню и взаимодействие с пользователем 11

5 Реализация операций с базой правил 12

5.1 Добавление правила 12

5.2 Изменение правила 12

5.3 Удаление правила 12

6 Тестирование программы 13

6.1 Проверка загрузки базы правил 13
6.2 Проверка CRUD-операций 13
6.3 Проверка прямой цепочки рассуждений 13
6.4 Анализ результатов 14
Заключение 14
ПРИЛОЖЕНИЕ А — ЛИСТИНГ ПРОГРАММЫ 15
ПРИЛОЖЕНИЕ Б — ФАЙЛ БАЗЫ ПРАВИЛ 15
ПРИЛОЖЕНИЕ В — ДОКУМЕНТАЦИЯ ПРОМПТОВ И ОТВЕТОВ 15

Введение

Экспертные системы являются одним из направлений искусственного интеллекта, в котором знания предметной области представляются в виде фактов и правил, а специальный механизм вывода позволяет получать новые факты на основании уже известных. Одним из вариантов такого вывода является прямая цепочка рассуждений.

Целью лабораторной работы является разработка программной оболочки экспертной системы, реализующей механизм прямой цепочки рассуждений, а также предоставляющей возможность работы с базой правил и рабочей базой данных.

В качестве предметной области выбрано формирование списка покупок для приготовления блюда. Программа получает исходные требования к блюду, последовательно применяет подходящие правила и формирует новые факты, в том числе факты о продуктах, которые необходимо приобрести.

В ходе работы были спроектированы структура экспертной системы, формат внешней базы правил, структуры данных для представления правил и рабочей базы данных, алгоритм прямой цепочки рассуждений, а также операции добавления, изменения и удаления правил.

1 Постановка задачи

Необходимо разработать оболочку экспертной системы на языке программирования Python, реализующую прямую цепочку рассуждений. База правил должна храниться отдельно от программного кода в текстовом файле и поддерживать редактирование.

Разрабатываемая система должна обеспечивать следующие возможности:

1) загрузка базы правил из отдельного текстового файла;
2) проверка корректности правил при загрузке и сохранении;
3) добавление нового правила;
4) изменение существующего правила;
5) удаление правила;
6) просмотр базы правил;
7) задание начальной ситуации рабочей базы данных;
8) просмотр текущего состояния рабочей базы данных;
9) запуск прямой цепочки рассуждений;
10) вывод информации о сработавших правилах и полученных новых фактах.

2 Описание предметной области

2.1 Выбор предметной области

Предметной областью является формирование списка покупок для приготовления блюда. Пользователь задаёт требования к блюду, например, необходимость быстрого, недорогого, полезного, мясного или вегетарианского блюда, а также указывает приём пищи. На основании этих исходных фактов экспертная система последовательно получает промежуточные выводы и конкретные продукты.

Такая предметная область удобна для демонстрации прямой цепочки рассуждений, поскольку один исходный факт может приводить к появлению промежуточного факта, который затем становится условием другого правила. В результате образуется цепочка вывода.

2.2 Принцип работы экспертной системы

Работа системы начинается с формирования рабочей базы данных, содержащей начальные факты. Затем программа последовательно просматривает базу правил. Если все условия некоторого правила присутствуют в рабочей базе данных, правило срабатывает, а его заключение добавляется в рабочую базу данных. После появления нового факта правила проверяются повторно.

Процесс продолжается до тех пор, пока очередной проход по базе правил не приведёт к появлению новых фактов. Таким образом, система автоматически строит цепочку вывода от исходной ситуации к конечным заключениям.

3 Проектирование экспертной системы

3.1 Архитектура системы

Программа разделена на несколько логических компонентов. Такое разделение позволяет отделить пользовательский интерфейс, хранение и редактирование правил, механизм логического вывода и сами знания предметной области.

```
Lab1/
├── main.py
├── rules_manager.py
├── inference.py
└── rules.txt
```

Модуль main.py отвечает за взаимодействие с пользователем и главное меню. Модуль rules_manager.py содержит класс правила, функции проверки, загрузки, сохранения и редактирования правил. Модуль inference.py реализует прямую цепочку рассуждений. Файл rules.txt является внешней базой знаний.

3.2 Представление правил

Правило представляется объектом класса Rule и состоит из идентификатора, списка условий и заключения. В текстовой базе используется следующий формат:

R03 | подходит_курица & нужно_недорогое_блюдо | нужен_недорогой_мясной_продукт

Знак «&» используется для объединения нескольких условий. Правило может сработать только в том случае, если каждое его условие присутствует в рабочей базе данных.

```
class Rule:
def __init__(self, rule_id, conditions, conclusion):
    self.rule_id = rule_id
    self.conditions = conditions
    self.conclusion = conclusion
```

3.3 Рабочая база данных

Для хранения текущих фактов используется множество set. Такой выбор позволяет исключить дублирование фактов и быстро проверять наличие условия в рабочей базе.

```
working_memory = set()
```

Пример начальной ситуации:

```
working_memory = {
 "нужен_ужин",
 "нужно_быстрое_блюдо",
 "нужно_недорогое_блюдо",
 "нужно_мясное_блюдо"
}
```

3.4 База знаний

База знаний содержит 15 правил. Они образуют несколько цепочек вывода: для ужина, завтрака и обеда, а также для получения конкретных продуктов.

Начальные факты системы:

нужен_завтрак

нужен_обед

нужен_ужин

нужно_быстрое_блюдо

нужно_недорогое_блюдо

нужно_полезное_блюдо

нужно_мясное_блюдо

нужно_вегетарианское_блюдо

База правил:

R01 | нужен_ужин & нужно_быстрое_блюдо | подходит_простое_блюдо
R02 | подходит_простое_блюдо & нужно_мясное_блюдо | подходит_курица
R03 | подходит_курица & нужно_недорогое_блюдо | нужен_недорогой_мясной_продукт
R04 | подходит_курица & нужен_недорогой_мясной_продукт | купить_куриную_грудку
R05 | подходит_простое_блюдо | нужен_гарнир
R06 | нужен_гарнир & нужно_недорогое_блюдо | подходит_рис
R07 | подходит_рис | купить_рис

R08 | нужен_ужин & нужно_полезное_блюдо | нужны_овощи
R09 | нужны_овощи | купить_овощи
R10 | нужен_завтрак & нужно_быстрое_блюдо | нужны_яйца
R11 | нужны_яйца | купить_яйца
R12 | нужен_завтрак & нужно_полезное_блюдо | нужны_фрукты
R13 | нужны_фрукты | купить_фрукты
R14 | нужен_обед & нужно_вегетарианское_блюдо | подходит_макароны
R15 | подходит_макароны | купить_макароны

4 Разработка программной части

4.1 Структура программы

Программа состоит из трёх программных модулей и отдельного файла базы правил. Такое построение соответствует требованию отделения базы знаний от программного модуля.

Назначение компонентов:

- main.py — главное меню, ввод начальной ситуации, вывод информации и вызов функций системы;
- rules_manager.py — класс Rule, проверка правил, загрузка, сохранение и CRUD-операции;
- inference.py — проверка условий и выполнение прямой цепочки рассуждений;
- rules.txt — внешняя текстовая база правил.

4.2 Модуль управления правилами

В модуле rules_manager.py реализованы функции получения, добавления, изменения и удаления правил. Перед добавлением и сохранением выполняется проверка корректности данных.

Проверяется наличие идентификатора, отсутствие дублирования идентификаторов, наличие хотя бы одного условия и наличие заключения. При загрузке дополнительно проверяется формат строки.

4.3 Загрузка и сохранение базы правил

Функция load_rules() открывает файл rules.txt, игнорирует пустые строки и комментарии, разбирает остальные строки по разделителю «|», а условия — по символу «&». Функция save_rules() выполняет обратную операцию и записывает текущую базу правил в файл.

При обнаружении ошибки формата программа формирует сообщение с номером строки и описанием проблемы. Это позволяет обнаруживать ошибки в базе знаний до запуска механизма вывода.

4.4 Алгоритм прямой цепочки рассуждений

Механизм вывода реализован в функции forward_chaining(). Сначала проверяется возможность срабатывания каждого правила. Если все его условия выполнены и заключение ещё отсутствует в рабочей базе, оно добавляется как новый факт.

```
        while changed:
changed = False

for rule in rules:
    if can_fire(rule, working_memory):
        if rule.conclusion not in working_memory:
            working_memory.add(rule.conclusion)
            changed = True
```

После прохода по всем правилам цикл повторяется, если был добавлен хотя бы один новый факт. Если новых фактов больше нет, цепочка рассуждений завершается.

4.5 Главное меню и взаимодействие с пользователем

Пользователь взаимодействует с программой через текстовое меню:

1. Задать начальную ситуацию
2. Показать рабочую базу данных
3. Запустить прямую цепочку рассуждений
4. Показать базу правил
5. Добавить правило
6. Изменить правило
7. Удалить правило
8. Очистить рабочую базу данных
0. Выход

Работа с базой правил и рабочей базой данных выполняется независимо. Изменение rules.txt не требует изменения исходного кода программы.

5 Реализация операций с базой правил

5.1 Добавление правила

Для добавления пользователь задаёт идентификатор правила, условия и заключение. Перед помещением правила в список выполняется проверка, что такой идентификатор ещё не используется и что правило содержит необходимые элементы.

5.2 Изменение правила

Для изменения пользователь указывает идентификатор существующего правила, после чего вводит новый набор условий и новое заключение. Идентификатор сохраняется, а содержимое правила заменяется.

5.3 Удаление правила

Удаление выполняется по идентификатору. Если правило найдено, оно удаляется из списка и база правил сохраняется в текстовый файл.

6 Тестирование программы

6.1 Проверка загрузки базы правил

При запуске программа загружает rules.txt. Для проверки загрузки использовалась база из 15 правил. Каждая строка преобразуется в объект Rule, а условия сохраняются в виде списка строк.

6.2 Проверка CRUD-операций

Были предусмотрены проверки основных операций с базой правил: добавления нового правила, изменения существующего правила и его удаления. Дополнительно проверяется обработка ошибок: повторного идентификатора, отсутствующего условия, отсутствующего заключения и некорректного формата строки.

6.3 Проверка прямой цепочки рассуждений

Для проверки был рассмотрен сценарий недорогого быстрого мясного ужина. В рабочую базу помещаются факты:

нужен_ужин
нужно_быстрое_блюдо
нужно_недорогое_блюдо
нужно_мясное_блюдо

В результате последовательного применения правил получаются следующие факты:

подходит_простое_блюдо
подходит_курица
нужен_недорогой_мясной_продукт
купить_куриную_грудку
нужен_гарнир
подходит_рис
купить_рис

Таким образом, демонстрируется многошаговый вывод: заключение одного правила становится условием следующего правила. Это соответствует принципу прямой цепочки рассуждений.

6.4 Анализ результатов

В результате проектирования и реализации получена программа, в которой база правил отделена от исходного кода, рабочая база данных может задаваться пользователем, а механизм вывода последовательно расширяет её новыми фактами. Реализованные CRUD-операции позволяют изменять базу знаний без изменения программного кода.

При отсутствии правила, заключение которого ещё не содержится в рабочей базе данных, алгоритм завершает работу. Если за весь запуск не было добавлено ни одного нового факта, программа сообщает, что ни одно правило не сработало.

Заключение

В ходе лабораторной работы была разработана оболочка экспертной системы на языке Python, реализующая прямую цепочку рассуждений. В качестве предметной области выбрано формирование списка покупок для приготовления блюда.

В процессе выполнения работы были разработаны класс Rule и структуры хранения базы правил и рабочей базы данных, реализованы загрузка и сохранение правил в отдельном текстовом файле, проверка корректности правил, операции добавления, изменения и удаления правил, а также алгоритм прямой цепочки рассуждений.

Разработанная структура позволяет отделить знания предметной области от программной логики. Это делает систему более удобной для дальнейшего расширения: в базу можно добавлять новые правила, изменять существующие связи и формировать новые сценарии вывода без изменения основного алгоритма.

ПРИЛОЖЕНИЕ А — ЛИСТИНГ ПРОГРАММЫ

Ссылка программы на GitHub: AI-LSTU/lab1.rar at main · belozertsevdm/AI-LSTU

ПРИЛОЖЕНИЕ Б — ФАЙЛ БАЗЫ ПРАВИЛ

Содержимое файла rules.txt:

R01 | нужен_ужин & нужно_быстрое_блюдо | подходит_простое_блюдо
R02 | подходит_простое_блюдо & нужно_мясное_блюдо | подходит_курица
R03 | подходит_курица & нужно_недорогое_блюдо | нужен_недорогой_мясной_продукт
R04 | подходит_курица & нужен_недорогой_мясной_продукт | купить_куриную_грудку
R05 | подходит_простое_блюдо | нужен_гарнир
R06 | нужен_гарнир & нужно_недорогое_блюдо | подходит_рис
R07 | подходит_рис | купить_рис
R08 | нужен_ужин & нужно_полезное_блюдо | нужны_овощи
R09 | нужны_овощи | купить_овощи
R10 | нужен_завтрак & нужно_быстрое_блюдо | нужны_яйца
R11 | нужны_яйца | купить_яйца
R12 | нужен_завтрак & нужно_полезное_блюдо | нужны_фрукты
R13 | нужны_фрукты | купить_фрукты
R14 | нужен_обед & нужно_вегетарианское_блюдо | подходит_макароны
R15 | подходит_макароны | купить_макароны

ПРИЛОЖЕНИЕ В — ДОКУМЕНТАЦИЯ ПРОМПТОВ И ОТВЕТОВ

запрос: «Предложи предметную область для экспертной системы с прямой цепочкой рассуждений. Опиши базу правил для этой области.»

ответ: Были предложены варианты предметных областей и для каждой описана возможная база правил.

запрос: «Эта предметная область меня не устраивает - предложи что-нибудь на тему покупки продуктов в супермаркете»

ответ: Были предложены варианты экспертных систем, связанных с покупкой продуктов; наиболее подходящим вариантом была предложена система формирования списка покупок для приготовления блюда.

запрос: «Экспертная система формирования списка покупок для приготовления блюда показалась мне наиболее интересной. Помоги спроектировать структуру программы экспертной системы на Python. Требования: отдельный текстовый файл с правилами, прямая цепочка рассуждений, возможность добавления, удаления и изменения правил. Также должны быть средства для отображения состояния рабочей базы данных и как задать стартовую ситуацию - начальное состояние рабочей базы данных.»

ответ: Была спроектирована архитектура Python-проекта с `main.py`, `rules_manager.py`, `inference.py` и отдельным `rules.txt`. Были определены рабочая база данных, загрузка/сохранение правил, CRUD-операции и алгоритм прямой цепочки рассуждений.

запрос: «определить набор исходных фактов, промежуточных фактов и итоговых выводов, а затем построить из них 15 правил с настоящими цепочками рассуждений.»

ответ: Были определены исходные, промежуточные и итоговые факты и построено 15 правил, образующих последовательные цепочки вывода для формирования списка покупок.

запрос: «Проверь эти 15 правил на противоречия и покрытие сценариев»

ответ: Были проверены правила на прямые противоречия и покрытие сценариев. Было установлено, что прямых конфликтов между одинаковыми условиями с разными выводами нет; также были указаны места, которые можно улучшить для более связных цепочек.

запрос: «Пересобери все эти правила предложенным образом.»

ответ: Набор из 15 правил был перестроен с учётом предложенных исправлений, сохранив цепочки вывода от исходных требований к конкретным продуктам.

запрос: «Оставим предложенную архитектуру и перейдем к оформлению правил»

ответ: Был оформлен формат `rules.txt`: идентификатор правила, условия через `&` и заключение через `|`. Также было закреплено отделение базы правил от начальной рабочей базы данных.

запрос: «спроектировать структуры данных Python (`Rule`, база правил и рабочая БД) и механизм загрузки/сохранения `rules.txt`»

ответ: Были спроектированы класс `Rule`, список правил как база правил и `set` для рабочей базы данных. Также были разработаны `load_rules()` и `save_rules()` с проверкой формата, дубликатов ID, отсутствующих условий и заключений.

запрос: «выведи архитектуру текущей программы (того, какие модули имеем сейчас) и сам код этих модулей»

ответ: Была описана текущая архитектура из `main.py`, `rules_manager.py`, `inference.py` и `rules.txt` и приведён код основных модулей: управления правилами, загрузки/сохранения базы и алгоритма прямой цепочки.

запрос: «уже есть модуль управления правилами (без CRUD), главный (вывод на экран) и алгоритм цепочки. нужно добавить crud в rules manager и менюшку в main. как этотсделать?»

ответ: Были добавлены функции `get_rule()`, `add_rule()`, `edit_rule()` и `delete_rule()` в `rules_manager.py`, а в `main.py` предложено меню для задания начальной ситуации, просмотра рабочей БД и правил, запуска вывода, CRUD-операций и очистки рабочей БД.